With the rapid advancement of technology, to what extent is it possible to improve the safety of public-key encryption systems?

Word Count: 3'964

Index

1.0 Introduction

Data protection is a very serious problem. Cryptography has been one of the most effective solutions. Its first recorded use was by the Spartans, who communicated in code between military commanders [1]. They used manual cryptography, based on mnemonic devices using words to convey a message different from their actual meaning, to communicate different messages. These mnemonics could have been discovered, if so the messages would have lost their secrecy. This type of encryption was used until the 20th century, up until World War one [1]. After the First World War, cryptography began to be mechanised, using telephones and radio signals for encryption. An example of this was *Enigma,* an encryption system used by German troops during World War Two. This system relied on rotors. Each rotor changed an input letter to a different one a certain number of places down the alphabet. After each letter the rotor would turn, changing the encryption key. Once the first rotor had completed a full rotation, the second rotor would start and so on [13]. This seemed impossible to decipher as there were over 150 quintillion letter combinations possible. However even with the limited technology available at the time, a mathematician, Alan Turing, was able to develop a machine, called *Bombe,* to decipher messages encrypted using Enigma in 20 minutes [13]. Nowadays, according to computer scientist Scott Welch, the total computing capacity of all of the *Bombes* running for the full duration of the war was the equivalent of about 10 seconds of computing on a modern iPhone [14]. With developing technology decryption becomes easier, so encryption needs to advance.

As computers were introduced and data started to be digitalised, there was a need to create techniques to protect digital data.

During the 1970's an important cryptographic system was created, Rivet-Shamir-Adleman (RSA) [1]. This encryption system was considered very safe, and has been widely used to protect data. RSA is an example of asymmetric cryptography, meaning that different keys are used for encryption and decryption. The encryption key is public, whereas the decryption key is private and only known

to the receiver. RSA uses the product of large prime numbers to encrypt messages. To decrypt the messages, the two separate large numbers need to be found.

RSA usually used up to 1024-bit keys (around 308 digits). However, with the development of faster computers, security was no longer guaranteed. Due to many successful attacks, encryption using 2048-bit (around 616 digits) or longer keys is deemed necessary nowadays [2]. 2048-bit encryption is almost impossible to hack for current supercomputers. However, a quantum computer could decrypt the messages almost instantaneously [9]. Whereas the introduction of more powerful machines used to require longer keys, with quantum computers, longer keys seem to no longer be a remedy. This is worrying as quantum machines are more and more common, and Google Inc. plans to release a commercial quantum computer by this decade's end [3]. Public-key cryptography systems such as RSA or other current systems such as Diffie-Hellman are at risk of becoming unusable.

A natural question to ask would be: With the rapid advancement of technology, to what extent is it possible to improve the safety of public-key encryption systems ?

To answer this question we will focus on the mathematics behind RSA, to understand how its former strengths may no longer be strong enough. Next we will evaluate why more modern encryption systems such as elliptic curve cryptography are less vulnerable and could be a possible improvement.

2.0 Public-Key Cryptography

Public key cryptography (PKC) involves a public key and a private key (a *public key pair*). Each public key is published, but the corresponding private key is kept secret. After encryption using the public key, data can only be decrypted with the corresponding private key.

This type of cryptography not only enables data encryption and decryption, but also enables the "nonrepudiation" of data. Nonrepudiation prevents the sender from claiming that the data was never sent and prevents the data from being altered.

Figure 1.
Public-key encryption [15]

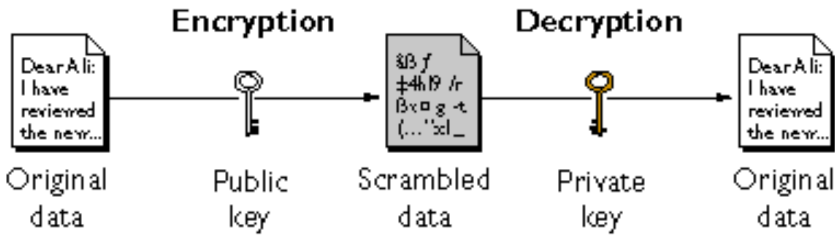


Figure 1. shows how public-key encryption works. Imagine a sender, called Bob, wants to send data to someone, called Alice, with a public key. The public key is available to anyone who wants to share data with Alice. Based on this key, calculations can be done to encipher the message. The private key is only available to the receiver, Alice. This key is the only way to decrypt the scrambled data and return it to the original data. For the process to work, Alice's private key must be the corresponding key to the public key used by Bob. The message will be deciphered incorrectly if the keys are not a corresponding pair.

The primary use of PKC is the creation of digital signatures, encrypted messages created using a user's private key, ensuring the user's identity. They are like signatures, recognisable and unique traits which help distinguish the user from other users. These signatures provide authentication, nonrepudiation and confidentiality. Because of these characteristics, many institutions use this type of encryption. The main fields in which it is used are banking and online transactions [4]. When

purchasing an item online, to ensure that only the organisation can see the transaction data, the data is encrypted using a public key specific to that customer. When the organisation receives the data, a private key is used to decipher and interpret it.

There are many algorithms which use PKC. The most commonly used ones are Rivest-Shamir-Adleman (RSA), Diffie-Hellman, Advanced Encryption Standard (AES), Data Encryption Standard (DES) and Elliptic Curve Cryptography (ECC). The next section will focus on the most popular one, RSA. It will consider why PKC systems have widely used it and what RSA's weaknesses are.

3.0 Rivest-Shamir-Adleman (RSA):

The RSA encryption system was first released to the public in 1977. It uses a type of "trapdoor function", namely the fact that it is straightforward to multiply large integers but very hard to factorise them. They are called "trapdoors" because they are easy to fall into but difficult to escape from, or in this context easy to compute but hard to reverse. For example, it is easy to find that 47 times 23 equals 1081, but finding the two factors of 1081 takes much more time. RSA uses very large prime numbers to encrypt messages so that data decryption by an eavesdropper involves prime factorisation. We need a mathematical ingredient, Euler's totient function, to use RSA encryption.

3.1 Euler's totient function

"Euler's totient function counts the number of positive integers less than N that are coprime to N. That is, $\phi(N)$ is the number of $m \in \mathbb{N}$ such that $1 \leq m < N$ and $\gcd(m, N) = 1$"[1]. In RSA encryption the number $\phi(N)$ is used for all calculations. This section will prove the formula:

$\phi(N) = N\left(1 - \frac{1}{p_1}\right) \dots \left(1 - \frac{1}{p_k}\right)$, where: $p_1, p_2, \dots p_k$ are the different prime numbers diving N.

To prove the formula, two properties of this function have to be stated:

- The function is multiplicative, only if a and b are coprime. This means that $\phi(ab) = \phi(a) \times \phi(b)$ if $gcd(a, b) = 1$ [16].
- The number N can be rewritten as a product of prime powers: $N = {p_1}^{e_1} \dots {p_k}^{e_k}$ by the Unique Prime Factorisation Theorem [16]. Therefore $\phi(N)$ can be expressed as the product of the $\phi({p_i}^{e_i})$, p prime, $e \in \mathbb{Z}^+$.

[1] Brilliant.org. *Euler's totient function*. Brilliant Math & Science Wiki. https://brilliant.org/wiki/eulers-totient-function/

Consider $\phi(p^e)$ where p is a prime number and $e \in \mathbb{Z}^+$. The positive integers in the range $[1, p^e]$, which are not coprime to p^e, are the multiples of p in that range. There are p^{e-1} of these since $p \times p^{e-1} = p^e$. Therefore $\phi(p^e) = p^e - p^{e-1} = p^e\left(1 - \frac{1}{p}\right)$.

With these two properties known:

If $N = {p_1}^{e_1} \ldots {p_k}^{e_k}$ and $e_i > 0$, then

$$\phi(N) = N\left(1 - \frac{1}{p_1}\right) \ldots \left(1 - \frac{1}{p_k}\right)$$ □

However, if all primes appear as a single factor, the function can be simplified further.

$$\phi(N) = N\left(1 - \frac{1}{p_1}\right) \ldots \left(1 - \frac{1}{p_k}\right)$$

$$= N\left(\frac{p_1 - 1}{p_1}\right) \ldots \left(\frac{p_k - 1}{p_k}\right)$$

$$= N\left(\frac{1}{(p_1 \ldots p_k)}\right)(p_1 - 1) \ldots (p_k - 1)$$

$$= N\left(\frac{1}{N}\right)(p_1 - 1) \ldots (p_k - 1)$$

$$= (p_1 - 1) \ldots (p_k - 1)$$

For RSA encryption, two primes, p and q, are used; therefore, the formula will be:

$$\phi(N) = (p - 1)(q - 1)$$ □

3.2 Steps for Encryption and Decryption

A series of steps must be followed to encrypt and decrypt messages using RSA encryption systems [5]. Firstly all the steps will be explained, and then an example will be shown. The first set of steps to be followed involves creating the public and private key pair.

I: Choose two prime numbers, p and q. These numbers are usually chosen of similar sizes such that their product is of the required bit length. Usually, the product's bit-length is 1024 bits or 2048 bits. Around 300 and 600 digits, respectively.

II: Compute the product of the numbers p and q.

$$N = pq.$$

N will be the modulus used in the encryption and decryption of the messages and is part of the public key.

III: Compute $\phi(N)$ using the formula presented in section 3.1:

$$\phi(N) = (p-1)(q-1)$$

IV: Choose the rest of the public key. The last number which needs to be chosen is the public exponent, referred to as the number e (not to be confused with Euler's number). The integer e should be $1 < e < \phi(N)$ such that $\gcd\left(e, \phi(N)\right) = 1$. This means that the selected integer will be coprime to $\phi(N)$. N and e constitute the public key and can be freely distributed.

V: Compute the private key d. This number d is the secret exponent. This integer should be: $1 < d < \phi(N)$ and satisfy the following:

$$ed \equiv 1 \bmod \phi(N) \;\Leftrightarrow\; d \equiv e^{-1} \bmod \phi(N)$$

The integer d is the inverse of e modulo $\phi(N)$.

VI: The key pair has been generated. The public key is (N, e), and the private key is (p, q, d).

Observe that to encrypt a message the calculations are done modulo N, whereas $\phi(N)$ is needed to find the secret exponent d. To compute $\phi(N)$ the two primes p and q are needed. This shows that for RSA to be effective the two primes need to remain secret.

Now that the key pair has been generated, the first process is to encrypt the message, which only involves one calculation. We represent it as a number representing the plaintext (unencrypted message) M, $0 < M < N - 1$.

The cypher text (encrypted message) can be referred to as C. To turn the plaintext M, into C, compute the following formula involving the public exponent *e:*

$$C \equiv M^e \bmod(N)$$

To decrypt the cypher text and return to the plaintext the formula uses the private exponent *d:*

$$M \equiv C^d \bmod(N)$$

This works because:

$$C^d \bmod(N) \equiv \left(M^e\right)^d \bmod(N) \equiv \left(M^{ed}\right) \bmod(N)$$

Recall that $ed \equiv 1 \bmod \phi(N)$. Then by definition of "mod", there is a number such that:

$$ed - 1 \equiv k \times \phi(N)$$

Rewrite:

$$M^{ed} \equiv M^{(ed-1)+1} \equiv MM^{(ed-1)} \equiv MM^{k\phi(N)} \bmod(N) \equiv M\left(M^{\phi(N)}\right)^k \bmod(N)$$

From Euler's theorem [24] it can be seen that:

$M^{\phi(N)} \bmod(N) \equiv 1$, observe that this only works if the exponent is $\phi(N)$.

Therefore:

$$M^{ed} \equiv M \times 1^k \bmod(N) \equiv M \bmod(N)$$

3.3 Worked example

These steps can be used to encrypt and decrypt any message. We provide a simple example, the encryption and decryption of the word "MATHS".

Step 1: Choose two prime numbers. For illustration purposes, the numbers chosen are a lot smaller than the numbers that would be used by enterprises.

$p = 13$

$q = 17$

Step 2: Compute the product of the prime numbers $N = pq$:

$N = 13 \times 17 = 221$

Step 3: Compute Euler's totient function, $\phi(N) = (p-1)(q-1)$:

$\phi(N) = (13-1)(17-1) = 12 \times 16 = 192$

Step 4: Choose the public exponent *e*. This number is coprime to and smaller than $\phi(N)$ (there is normally more than one possibility):

$e = 41$

Step 5: Generate the secret exponent *d*. The integer d is the inverse of *e* modulo $\phi(N)$[3]:

$d \equiv 41^{-1} \bmod (192) \equiv 89$

Step 6: Now the plaintext can be encrypted. When working with letters, ASCII code can be used to convert each letter into numbers. In Table 1. the numbers corresponding to each letter are shown.

Table 1.
Ascii code correspondence

Letters	Code
M	77
A	65
T	84
H	72
S	83

Each letter can be separately encoded and then the results can be put together[2]:

$C_M = 77^{41} \bmod(221) = 77$

$C_A = 65^{41} \bmod(221) = 156$

$C_T = 84^{41} \bmod(221) = 67$

[2] *Computed with* Wolfram Alpha, https://www.wolframalpha.com

$C_H = 72^{41} \bmod(221) = 89$

$C_S = 83^{41} \bmod(221) = 83$

Each letter can be separately sent and decrypted by the receiver using the secret exponent $d = 89$[2]:

$M_M = 77^{89} \bmod(221) = 77$

$M_A = 156^{89} \bmod(221) = 65$

$M_T = 67^{89} \bmod(221) = 84$

$M_H = 89^{89} \bmod(221) = 72$

$M_S = 77^{89} \bmod(221) = 83$

The receiver can then convert the numbers back to letters by using the ASCII conversion table.

3.4 Vulnerabilities

This cryptographic system has been in use for over 40 years and at the time when it was created it was considered the best and safest way to protect data. However, there have been many attacks on this system and a lot of these have been successful [6].

3.4.1 Brute force attack and Shor's algorithm

A brute force attack is when the attacker tries to factor *N*. A computer tries every possible division by a prime until it finds the correct one. *N* is a composite number (meaning it is an integer obtained by multiplying two smaller positive integers) and one factor of *N* has to be less or equal to $\sqrt{N}$.

Proof:

Consider a number $N = ab\,, \;\; 1 < a, b < N$

If both $a, b > \sqrt{N}$, then $ab > \sqrt{N} \times \sqrt{N} = N$

a contradiction since $ab = N$

This contradiction proves that *N* must have one factor less than or equal to $\sqrt{N}$. □

Therefore, a brute force attack will surely find a prime factor of N if he tries dividing N by all primes less or equal to $\sqrt{N}$. This means that a finite number of trials will surely lead to cracking the code.

Shor's algorithm is one of the fastest ways to compute the prime factors of a number [17]. This algorithm was created by mathematician Peter Shor in 1994. It requires a powerful quantum computer to be fully efficient [17]. With quantum computers becoming more and more common, this algorithm will pose serious threats to the integrity of RSA encrypted messages. This algorithm would be able to find the two prime factors almost instantaneously [9].

3.4.2 Key length

Nowadays, the keys N, used in RSA encryption systems are either 1024 bits or 2048 bits in length (around 300 and 600 digits respectively). When RSA was first introduced the key length was very short, only a few hundred bits. However, it still took 17 years to decrypt the first encrypted message [18]. At the pace at which technology is developed now, the key length would have to be much bigger to survive attacks. A bigger key size requires a longer time to do all the calculations needed to encrypt and decrypt the data and more storage capacity. According to NIST (National Institute of Standards and Technology), the minimum required key size since 2012 is 2048 bits [8]. Traditional machines cannot encrypt messages encrypted using this key length. For quantum machines, these messages are easy to decipher. According to cryptography expert Karen Martin, even an increased key size to 15,360 bits has little to no effect on the security against quantum computers [9].

3.4.3 Key implementation

To generate keys, RSA systems use something called a Cryptographic Secure Pseudorandom Number Generator (CSPRNG) [7]. These systems are "pseudorandom" as they can be configured to have certain limits or to generate keys in a specific range. These systems might sound safe at first,

as they are random generators, however, if many organisations use the same generators configured in the same way, the keys generated are less random. If an attacker can breach into one of the generators they can configure it in a way such that it makes it easier to then find the generated keys. For example, allowing only keys within a specific range. This would simplify the process of finding the different seeds that are produced. Moreover, if many enterprises are using the same CSPRNGs, the amount of data which is at risk of being exposed increases exponentially. A PhD student from Paris University found that there are many possible attacks on the Number Generators posing real threat to the integrity of data [10].

3.5 RSA Conclusion

RSA has been a useful and important PKC since its creation in 1977 and many factorisation problems introduced a long time ago still have not been solved. As most of the viable attacks rely on high numbers of trial and error, with better computational power the number of trials per unit of time will be much higher and therefore the right factors will be found much faster. This means that there is a need for better encryption systems, which have different techniques to encrypt data, and that can survive attacks from powerful machines such as quantum computers.

4.0 Elliptic Curve Cryptography (ECC)

In the 70s, two mathematicians, Victor Miller and Neal Koblitz, developed a new cryptographic system that uses elliptic curves as the basis for encryption [11]. Elliptic curve cryptography quickly gained interest as it required a much smaller key size for the same amount of security as RSA. In the early 2000s the National Security Agency (NSA), decided that ECC would be the Standard Suite B algorithm[3]. Like RSA, ECC relies on a "trapdoor function". ECC is commonly used together with other crypto systems. ECC is used to derive a shared secret key and then another crypto system, the most common one being Diffie-Hellman use the shared key for encryption.

4.1 Mathematics behind ECC

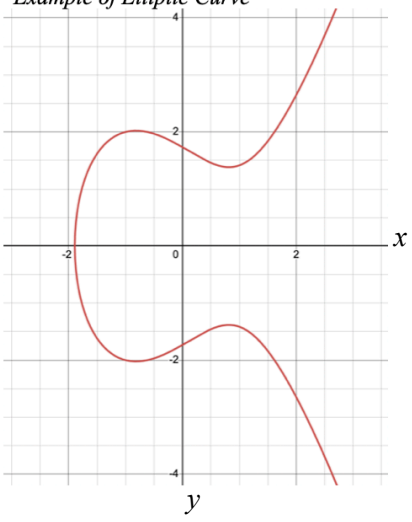
An elliptic curve *E(F)* is defined as a set of points over a finite field *F*[4] satisfying an equation of the form: $y^2 = x^3 + a\,x + b$

Figure 2 shows an example of an elliptic curve over the real numbers, $x, y \in \mathbb{R}$ with the equation:

$$y^2 = x^3 - 2x + 3$$

This curve has important properties that make it useful in cryptography. One of these is its reflection symmetry in the x-axis: For every point $P(x,\ y)$ on *E,* there is a point $P'(x,\ -y)$ on *E*. Another property is that any line intersects the elliptic curve in at most 3 distinct points.

Figure 2. [12]
Example of Elliptic Curve



[3] Standard suite B algorithms are algorithms suitable for protecting both classified and unclassified national security systems and sensitive governmental information.

[4] A finite field is a set with a finite number of elements, along with two operations defined on that set, an addition operation and a multiplication operation

These properties are used in cryptography to generate the keys for encryption and decryption of messages. Elliptic curves rely on point addition and scalar multiplication to produce keys. As in every PKC, there are public and private keys.

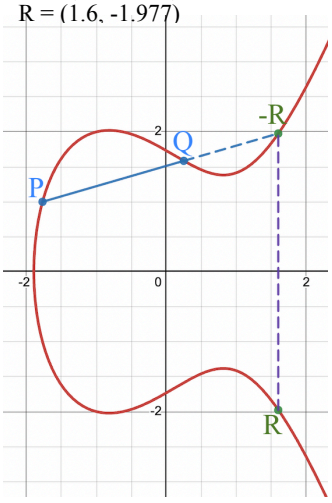
4.1.1 Point Addition

Elliptic curves are additive groups, which means that points on the curve form a set, and that the operation "addition" is defined for elements of the set. The addition of two points on an elliptic curve is defined geometrically [19]. We will explain it here.

Define the negative of a point $P(x_P, y_P)$ as its image under reflection in the x-axis, the point $-P(x_P, -y_P)$. For each point P on an elliptic curve, the point -P will be on the curve.

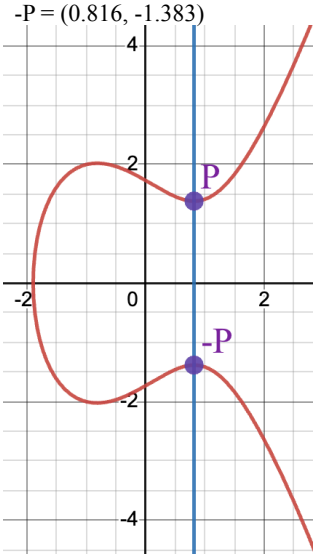
Consider two distinct points, P and Q, where $P \neq -Q$, on the curve E: $y^2 = x^3 - 2x + 3$. See Figure 3. To add the two points, a line is drawn through the points. The line drawn will intersect the curve in exactly one more point, call -R. This point is reflected in the x-axis to point R. This process defines addition on E: $P + Q = R$.

Figure 3.
Example of Point addition [12]
P = (-1.796, 1)
Q = (0.25, 1.586)
-R = (1.6, 1.977)
R = (1.6, -1.977)



If $P = -Q$, it means that point P is added to its reflection in the x-axis, point -P. The result will be equal to the point $\mathcal{O}$, which is a point at infinity. $P + (-P) = \mathcal{O}$. This is because the line drawn through the points will be a straight vertical line which won't intersect the curve at any other point. See Figure 4.

Figure 4.
Example of inverse addition [12]
P = (0.816, 1.383)
-P = (0.816, -1.383)



The last definition of point addition which is needed in ECC is doubling a certain point P. This means adding a point to itself. The process involves the line which is a tangent to the point. The tangent line will intersect the curve at one other point, which will be called -R and which is reflected in the

x-axis to R. This law defines: $P + P = 2P = R$. See Figure 5.

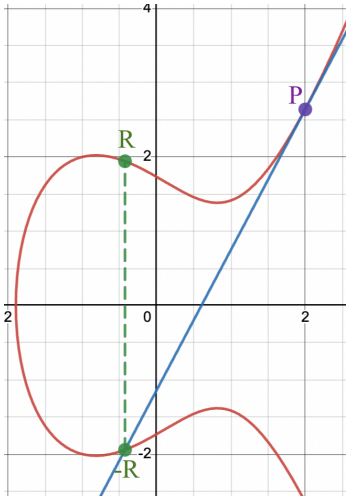


Figure 5.
Example of self addition [12]
P = (2, 2.639)
-R = (-0.425, -1.942)
R = (-0.425, 1.942)

4.1.2 Scalar multiplication

The second mathematical operation used in ECC is Scalar Multiplication. Scalar multiplication is the process of multiplying a certain point by k, or equivalently for $k \in \mathbb{Z}^+$, adding the point to itself k times. $kP = \underbrace{P + P + P + \ldots}_{k \text{ times}}$ [20].

For example if $k = 4$, $4P = P + P + P + P = 2P + P + P = 3P + P$. This is shown on the curve $y^2 = x^3 - 2x + 3$ using doubling and two additions in Figure 6. With $P = (2,\ 2.639)$, the result is $4P = (1,\ -1.414)$.

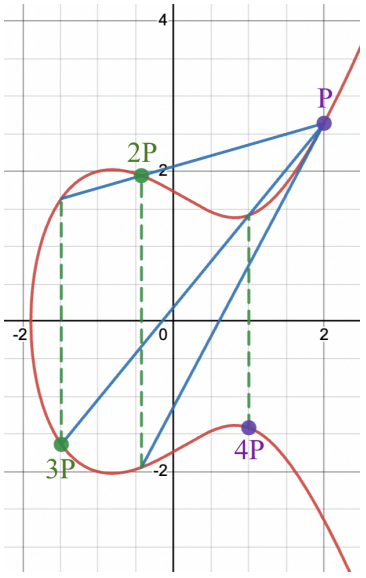


Figure 6.
Example of scalar multiplication [12]
P = (2, 2.646)
2P = (-0.425, 1.942)
3P = (-1.489, -1.636)
4P = (1, -1.414)

4.2 Steps for Encryption and Decryption

As for every PKC, the first steps involve the creation of private and public keys. For these steps we will have a sender, Alice, and a receiver Bob. ECC differs from RSA as it is an asymmetric encryption system.

I: Choosing a Generator point G(x, y) on the curve $y^2 = x^3 + ax + b$.

II: Creating Alice and Bob's private keys. Alice chooses a random positive integer as private key (k). Bob also chooses a private key (j), also a random positive integer.

III: Computing the public keys. Alice computes her public key (A) by scalar multiplying G by her private key: $A = kG$. Bob computes his public key (B) by scalar multiplying G by his private key: $B = jG$.

IV: Now Alice and Bob compute a shared secret point. To do so Alice multiplies Bob's public key (B) by her private key (k). $S_A = kB$. Bob multiplies Alice's public key (A) by his private key (j). $S_B = jA$.

Now:

$$S_A = kB = k(jG) = j(kG) = jA = S_B$$

V: The shared secret point is then used to derive a symmetric encryption key. To do this a Key Derivation Function (KDF) is needed. A KDF takes into consideration certain parameters such as key length in bits, to convert the shared secret point to a usable key by applying a one-way hash function. To be usable, normally, the key should be should be in hexadecimal or binary form. A one-way hash function is a mathematical function that takes an input and produces a fixed-size string of bits, typically a hexadecimal number.

VI: The derived key can then be used together with crypto systems such as DH to create a cypher text which will be really difficult to decrypt for a hacker.

VII: Deciphering the message. Bob can compute his shared secret point S_B, and since this point is equal to Alice shared secret point, when using the KDF, they will obtain the same key. This will allow Bob to easily decrypt the cypher text.

Observe that the strength relies on the fact that having a certain point A on the curve does not allow a hacker to find *k* or *G*. To find these two values, the correct line through *-A* would be needed but there an infinite amount of possibilities. Therefore, trial and error does not guarantee that a hacker will find the correct line, meaning that the process of cracking a certain elliptic curve cannot be optimised by the use of quantum computers. Moreover, if a hacker is in possession of the point *G* and of one of the secret points, it is very hard to find the private keys. The secret points can have been generated using an infinite amount of integer possibilities, as $k, j \in \mathbb{Z}^+$. This also means that quantum computers would not optimise the cracking of an elliptic curve.

4.3 Advantages of ECC

4.3.1 Shorter key size

A key, the shared point S ran through a KDF, of size 256-bits in ECC is equivalent, in the security it provides, to a key of size 3072-bits RSA and 10'000 times stronger, meaning there are 10'000 more possible key combinations, than a 2048-bits RSA [21]. Table 2 compares the key sizes of RSA and ECC.

Table 2.
Key size comparison RSA vs ECC [22]
Eg. A key of length 15'360 bits in RSA provides the same safety as a key of length 521 bits in ECC

RSA and Diffie-Hellman Key Size (bits)	Elliptic Curve Key Size (bits)
1024	160
2048	224
3072	256
7680	384
15360	521

ECC is also very fast for the high level of security it provides. Due to the smaller key size, less data has to be transmitted when communicating [21], and less data has to be stored, as the length of the key size determines the length of the encrypted message and computations.

4.3.2 The Discrete Logarithm Problem

To crack ECC, hackers have to run a series of algorithms, namely the Elliptic Curve Discrete Logarithm Problem (ECDLP). This is a really hard algorithm to compute which has no known solutions. Therefore the only way to crack it is to try out random numbers, of which there are infinitely many, whereas for RSA finitely many attempts will ultimately lead to success. However

for each bit size provides more options than RSA, making it difficult for a brute force attack to succeed. This means that ECC relies on the hardness of the underlying mathematical problem rather than the lack of efficient algorithms to break it.

5.0 Conclusion

In conclusion, with quantum computers and super computers becoming more easily accessible, a change in the crypto systems used will be needed. The most common crypto system, RSA, has been successful for over 50 years and will probably still be used until quantum computers become the norm among people. EPFL Professor Arjen Lenstra defines the security provided by RSA-2048 as "sea security": decrypting requires the same energy as bringing an entire sea to boil [23]. This is an enormous amount of energy. Therefore, with current machines RSA is still usable. For example, for a MacBook Air 2021, with an M1 chip, a fairly powerful processor, even RSA-20 is almost impossible to break and takes on average 48hours to break. But a very powerful computer can do this almost instantaneously.

With quantum computers, many more algorithms, such as Shor's algorithm will be widely accessible, and PKCs such as RSA will be useless to encode very sensitive messages, unless their key size increases enormously. Even today there are many weaknesses with RSA which, although they involve long processes, can be dangerous for the integrity of data encrypted with RSA.

That is why there is a need for new algorithms which are powerful enough to survive attacks from quantum computers. A viable option is ECC. Elliptic curves are a great alternative to current systems as they provide the same amount of security with much shorter key size and lower computational power. NSA already considered it a safe option for the future in the early 2000's and nowadays it has become one of the best and safest options to replace RSA.

6.0 Bibliography

[1] Simmons, G. J. (1999, July 26). *History of cryptology*. Encyclopædia Britannica. Retrieved February 10, 2023, from https://www.britannica.com/topic/cryptology/History-of-cryptology

[2] Goldberg, J. (2014, April 1). *Large even prime number discovered: 1password*. 1Password Blog. Retrieved April 16, 2023, from https://blog.1password.com/large-even-prime-number-discovered/#:~:text=The%20prime%20numbers%20in%20cryptography,(about%20616%20digit)%20products

[3] Castellanos, S. (2022, June 29). *Google aims for commercial-grade quantum computers by 2029*. The Wall Street Journal. Retrieved April 16, 2023, from https://www.wsj.com/articles/google-aims-for-commercial-grade-quantum-computer-by-2029-11621359156

[4] Applications of public key cryptography and functioning process. (2019, April 26). *International Journal of Innovative Technology and Exploring Engineering*, *8*(6S4), 480–482. https://doi.org/10.35940/ijitee.f1100.0486s419

[5] Setting up RSA. http://www.isg.rhul.ac.uk/static/msc/teaching/ic2/demo/40.htm

[6] Boneh, D. (1999). *Twenty years of attacks on the RSA cryptosystem - Stanford University*. https://crypto.stanford.edu/~dabo/pubs/papers/RSA-survey.pdf

[7]Kee, L. (2021, May 6). *Council post: RSA is dead - we just haven't accepted it yet*. Forbes. https://www.forbes.com/sites/forbestechcouncil/2021/05/06/rsa-is-dead---we-just-haventaccepted-ityet/

[8] *Polk, T. (2020, December 10). Security and Privacy Controls for Information Systems and Organizations. CSRC. https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final*

[9] Martin, K. (2019, January 22). *Waiting for quantum computing: Why encryption has nothing to worry about*. TechBeacon. https://techbeacon.com/security/waiting-quantum-computing-why-encryption-has-nothing-worry-about#:~:text=AES-128%20and%20RSA-2048,little%20effect%20against%20quantum%20attacks.

[10] Martinez, F. (2021, September 17). Attacks on pseudo-random number generators hiding a linear structure - IACR. Cryptology ePrint Archive. https://eprint.iacr.org/2021/1204.pdf

[11] Wohlwend, J. (2016, April 18). *Elliptic curve cryptography: Pre and post quantum - MIT mathematics*. ELLIPTIC CURVE CRYPTOGRAPHY: PRE AND POST QUANTUM. https://math.mit.edu/~apost/courses/18.204-2016/18.204_Jeremy_Wohlwend_final_paper.pdf

[12] *Desmos | Graphing Calculator*. Desmos. Retrieved June 19, 2023, from https://www.desmos.com/calculator

[13] Furness, H. (2021, September 20). *How long would it take you to crack the Enigma machine*. How Long Would It Take You To Crack The Enigma Machine. https://the-axiom.uk/how-long-would-it-take-you-to-crack-the-enigma-machine/

[14] Welch, S. (2018, September 24). *How long would it take today's computers to crack the Enigma machine?* How long would it take today's computers to crack the Enigma Machine? https://qr.ae/pyx6pR

[15] IBM. (2021, March 3). *Public key cryptography*. IBM. Retrieved May 1, 2023, from https://www.ibm.com/docs/en/ztpf/1.1.0.15?topic=concepts-public-key-cryptography

[16] Dong, C. (2012, April 10). *Math in Network Security: A Crash Course*. Euler's totient function and Euler's theorem. https://www.doc.ic.ac.uk/~mrh/330tutor/ch05s02.html

[17] Rakhade, K. (2022, June 24). *Shor's Algorithm (for dummies)*. Medium. https://kaustubhrakhade.medium.com/shors-factoring-algorithm-94a0796a13b1

[18] Du Sautoy, M. (2003a). The music of the primes: Why an unsolved problem in mathematics matters. Harper Perennial.

[19] Blackberry. (2016). *Certicom: Elliptic curves*. 2.1 elliptic curve addition: A geometric approach. https://www.certicom.com/content/certicom/en/21-elliptic-curve-addition-a-geometric-approach.html#:~:text=P%20%2B%20Q%20%3D%20R%20is%20the,(xP%2C-yP).

[20] Lozano-Robledo, Prof. A. (1960, August 1). *Order of a point on an elliptic curve*. Mathematics Stack Exchange. https://math.stackexchange.com/questions/704202/order-of-a-point-on-an-elliptic-curve

[21] Thayer, W. (2014, June 10). *Benefits of elliptic curve cryptography*. PKI Consortium. https://pkic.org/2014/06/10/benefits-of-elliptic-curve-cryptography/

[22] Central Security Service. (2009, January 15). *The Case for Elliptic Curve Cryptography*. National Security Agency. https://archive.ph/archive.ph

[23] Lenstra, A. K., Kleinjung, T., & Thomé, E. (1970, January 1). Universal security; from bits and MIPS to pools, lakes -- and beyond. Cryptology ePrint Archive. https://eprint.iacr.org/2013/635

[24] San Jose State University. (2018). *The Mathematics behind RSA*. http://www.cs.sjsu.edu/~stamp/CS265/SecurityEngineering/chapter5_SE/RSAmath.html#:~:text=In%20RSA%2C%20we%20have%20two,the%20private%20key%20is%20d.